



Verification of a Simple ALU Design

Using the cocotb Verification Environment

Report by:

Youssef Bassem Lamey

Submitted to:

Eng. Mohammed El-Adawy

ADI Summer Internship 2026 — Digital Design and Verification Team

1. Objective

This report documents the functional verification of a simple 8-bit signed Arithmetic Logic Unit (ALU) using the cocotb Python-based verification framework. The goal was to build a lightweight, self-checking testbench that exercises all four supported ALU operations — addition, subtraction, bitwise AND, and bitwise XOR — and to confirm correct functional behavior of the design under test (DUT) through simulation.

2. Design Under Test (DUT)

The DUT is a synchronous ALU described in SystemVerilog. It accepts two signed 8-bit operands and a 2-bit opcode, and produces a registered signed 8-bit result along with status flags. Key characteristics of the design are summarized below.

Parameter	Description
Module name	ALU
Inputs	clk, rst_n, A [7:0] signed, B [7:0] signed, opcode [1:0]
Outputs	ALU_OUT [7:0] signed, carry_flag, arith_flag, logic_flag, zero_flag
Opcode encoding	0 = ADD, 1 = SUB, 2 = AND (bitwise), 3 = XOR (bitwise)
Reset behavior	Asynchronous, active-low (negedge rst_n). All registered outputs clear to 0.
Timing	All outputs are registered; results are valid one clock cycle after the operands/opcode are applied.

3. Verification Environment

The testbench was written in Python using cocotb, which drives and samples the DUT's signals directly through simulator callbacks rather than requiring a separate SystemVerilog testbench layer. The environment consists of a single cocotb test, `tb_top`, which:

- Starts a 10 ns period clock on `dut.clk`.
- Holds the design in reset (`rst_n = 0`) until the first falling edge of the clock, then releases reset.
- Sequentially runs two stimulus coroutines, `arith_stimulus` and `logic_stimulus`, each of which drives two test cases with randomized operand values.
- Samples `ALU_OUT` and the relevant status flags after each `RisingEdge` using a `ReadOnly` synchronization point, ensuring the registered outputs are stable before being read.

A deliberate design decision was to run the two stimulus coroutines sequentially (`arith_stimulus` followed by `logic_stimulus`) rather than concurrently. Both coroutines drive the same shared DUT signals (`A`, `B`, `opcode`); running them in parallel via `cocotb.start_soon` would create a race condition where whichever coroutine's assignment lands later in a given simulation timestep silently overwrites the other's stimulus, corrupting both test cases. Sequencing them removes this hazard entirely.

3.1 Opcode Coverage Strategy

Rather than picking opcodes purely at random (which risks repeating the same opcode across both passes of a category and leaving the other opcode untested), each stimulus coroutine uses `random.sample()` to generate a random ordering of its two opcodes with no repeats. This guarantees that:

- `arith_stimulus` exercises ADD and SUB, each exactly once, in random order.
- `logic_stimulus` exercises AND and XOR, each exactly once, in random order.

Operand values A and B are independently randomized within the full signed 8-bit range (-128 to 127) for every test case, giving each run a different, unpredictable stimulus set while still guaranteeing full opcode coverage across the four supported operations.

3.2 Test Sequence Code

The core stimulus logic is shown below.

```
async def arith_stimulus(dut):
    opcodes = random.sample(ARITH_OPCODES, k=len(ARITH_OPCODES))
    for opcode in opcodes:
        await FallingEdge(dut.clk)
        dut.A.value = random.randint(-128, 127)
        dut.B.value = random.randint(-128, 127)
        dut.opcode.value = opcode
        await RisingEdge(dut.clk)
        await ReadOnly()
        cocotb.log.info(f"[ARITH] opcode={opcode} ...")
```

`logic_stimulus` follows the same structure, iterating over the logic opcodes (AND, XOR) instead.

4. Simulation Results

The testbench was run to completion in simulation. All four test cases (2 arithmetic, 2 logic) executed without error, and the DUT's outputs matched the expected opcode-specific behavior in each case. The final simulation log and cocotb regression summary are shown below.

```
20.00ns INFO test [ARITH] opcode=0 A=36 B=43 ALU_OUT=79 carry_flag=0 arith_flag=1
30.00ns INFO test [ARITH] opcode=1 A=41 B=11 ALU_OUT=30 carry_flag=0 arith_flag=1

40.00ns INFO test [LOGIC] opcode=3 A=-93 B=-17 ALU_OUT=76 logic_flag=1
50.00ns INFO test [LOGIC] opcode=2 A=-42 B=-61 ALU_OUT=-62 logic_flag=1
50.00ns INFO test End sim
50.00ns INFO cocotb.regression ALU_tb.tb_top passed
50.00ns INFO cocotb.regression
*****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** ALU_tb.tb_top PASS 50.00 0.00 39353.57 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0 50.00 0.00 29508.26 **
*****
```

The regression summary confirms a single test, `ALU_tb.tb_top`, completed with status PASS, with TESTS=1 PASS=1 FAIL=0 SKIP=0 over a total simulated time of 50.00 ns. The log lines for the two logic test cases (opcode=3 / XOR and opcode=2 / AND) show correctly computed `ALU_OUT` values and the `logic_flag` asserted in both cases, consistent with the DUT's expected behavior for bitwise operations.

5. Conclusion

The cocotb-based verification environment successfully exercised all four opcodes of the ALU (ADD, SUB, AND, XOR) with randomized operands, and the DUT produced correct results and status flags for every test case. The sequential coroutine structure avoided signal-driving race conditions between the two stimulus generators, and the use of `random.sample()` for opcode selection guaranteed complete opcode coverage within each simulation run despite the small number of test cases. This exercise served as an introductory demonstration of building a self-checking cocotb testbench, including clock generation, reset sequencing, edge-synchronized signal driving, and read-only result sampling.